Faster Polynomial-Space Quantum Algorithms for Colouring, Domination and Monotone Local Search

Sebastian Senge
Independent Researcher

Abstract

We give quantum algorithms for exponential-time problems on n vertices or elements with bounded error, polynomial space and no quantum random access memory. Our results are these:

- 4-Colouring can be solved in quantum time $O(1.3032^n)$, from the polynomial-space branching algorithm of [43] with terminal algorithms of [45]. With [43] alone the bound is $O(1.3100^n)$.
- For $k \in \{5, 7, 8, 9, 10, 11\}$ and $13 \leq k \leq 18$, k-Colouring can be solved in quantum time below the previous best bound in this model to our knowledge, from $O(1.4518^n)$ for $k = 5$ to $O(1.9233^n)$ for $k = 18$.
- Dominating Set can be solved in quantum time $O(1.2235^n)$, Independent Dominating Set in $O(1.1583^n)$, and Edge Dominating Set and Minimum Maximal Matching in $O(1.1501^n)$.
- A quantum form of monotone local search: if the extension problem has a backtracking tree with c^k nodes up to polynomial factors, the problem is solved in quantum time $(2 - 1/c)^{n/2}$ up to polynomial factors. This gives Feedback Vertex Set in $O(1.3081^n)$, Subset and Group Feedback Vertex Set in $O(1.3229^n)$, Cluster Vertex Deletion in $O(1.2033^n)$, 3-Hitting Set in $O(1.2405^n)$, Proper Interval Vertex Deletion in $O(1.3541^n)$, Multicut on Trees in $O(1.1647^n)$ and Min-Ones d-SAT in $O^*((2-1/d)^{n/2})$. The approximate form gives quantum exponential-time approximations below the quantum exact bounds, such as a 1.1-approximation for Vertex Cover in $O(1.0555^n)$ and for Feedback Vertex Set in $O(1.2161^n)$.
- The exponents for k-colouring with $k \geq 6$ come from an evaluation of the reduction recursion whose values bound the running time of an explicit algorithm, computed in floating point with a margin.

To our knowledge, the previous best bounds in this model are $O(1.3231^n)$ for $k = 4$ and $O(1.4695^n)$ to $O(1.9366^n)$ for the other values of k above [36], and $O(2^{n/2})$ for Dominating Set [42]. For the other domination problems we know of no stated quantum bound; the walk on the enumeration of maximal independent sets gives $O^*(3^{n/6}) \subset O(1.2010^n)$.

1 Introduction

Deciding whether a graph has a proper colouring with k colours is one of the basic NP-complete problems. For $k = 3$ the algorithm of [5] is a branching algorithm in polynomial space with running time $O^*(1.3289^n)$. For $k = 4$ the algorithm of [43] is a branching algorithm in polynomial space with running time $O(1.7159^n)$. The recent work [45] gives $O^*(1.3204^n)$ and $O^*(1.6975^n)$, with dynamic programming over tree decompositions, which uses exponential space. For larger k, colouring reduces to smaller k through maximal independent sets [27, 8] and through balanced splits of the colour classes.

Quantum search applies to branching algorithms, as noted in [19], and gives the square root of the classical time of [5], so 3-colouring in quantum time $O(1.1528^n)$. The work [36] combines this with the reductions above and gives polynomial-space quantum algorithms without quantum

random access memory for every $k \leq 20$ below $O(2^n)$. Their Theorem 9 takes the exponent of 3-colouring as input. With quantum random access memory, the chromatic number can be computed in time $O(1.7956^n)$ [20], which is outside the model of this paper.

For domination problems, the best polynomial-space classical algorithms are branch and reduce algorithms analysed by measure and conquer [33, 34, 7, 6, 24, 44]. For Dominating Set, a quantum bound of $O(2^{n/2})$ queries was stated in [42]. For many vertex deletion and hitting set problems the best classical bounds come from monotone local search [17], which samples part of a solution at random and completes it with a parameterized algorithm.

Our contribution. We show that the 4-colouring algorithm of [43] is a backtracking tree in the sense of [31], which gives 4-colouring in quantum time $O^*(1.7159^{n/2}) \subset O(1.3100^n)$. The terminal algorithms of [45] that use no dynamic programming over decompositions keep this form, and with 3-colouring from [5] they give at most $p(n)\,1.6981^n$ nodes, so quantum time $O^*(1.6981^{n/2}) \subset O(1.3032^n)$. We then evaluate the recursion of [36] with 4-colouring as a second base case. The evaluation bounds the recursion from above on pieces of a partition, with all size limits rounded up to a grid, and we show that the algorithm can be run with these rounded limits. So the values are upper bounds on the time of an explicit algorithm. Table 1 lists the results. In Section 5 we show that the polynomial-space algorithms of [33, 34, 7] for domination problems are backtracking trees in their decision form, which gives the bounds of Table 2. In Section 6 we give a quantum form of monotone local search, with amplitude amplification over the samples and the walk of [31] on the extension algorithm, and apply it to the problems of Table 3.

k	Previous best bound [36]	With [43] alone	This paper, with [43, 45]
4	1.3231^n	1.3100^n	$\mathbf{1.3032^n}$
5	1.4695^n	1.4578^n	$\mathbf{1.4518^n}$
7	1.6511^n	1.6418^n	$\mathbf{1.6369^n}$
8	1.6585^n	1.6481^n	$\mathbf{1.6427^n}$
9	1.7460^n	1.7383^n	$\mathbf{1.7343^n}$
10	1.7775^n	1.7679^n	$\mathbf{1.7629^n}$
11	1.7775^n	1.7726^n	$\mathbf{1.7726^n}$
13	1.8499^n	1.8442^n	$\mathbf{1.8413^n}$
14	1.8580^n	1.8531^n	$\mathbf{1.8503^n}$
15	1.9303^n	1.9225^n	$\mathbf{1.9183^n}$
16	1.9303^n	1.9226^n	$\mathbf{1.9201^n}$
17	1.9303^n	1.9242^n	$\mathbf{1.9217^n}$
18	1.9366^n	1.9279^n	$\mathbf{1.9233^n}$

Table 1: Quantum time for k-colouring with bounded error, polynomial space and no quantum random access memory. All values are rounded up. The previous bounds are the best known to us. For $k = 3, 6, 12, 19, 20$ the bounds of [36] stand.

Scope. The last column of Table 1 uses terminal algorithms of [45], an arXiv paper that is, to our knowledge, unrefereed. The column with [43] alone does not depend on it. The bound $O^*(1.3217^n)$ for 3-colouring in the arXiv paper [29] is not used. Its linear program, as printed, has a larger optimum [45, Appendix F], and Remark 3.3 states what follows from that program. For the rows $k = 4, 5$, the arithmetic of the closed forms is checked in Lean. The rows $k = 6$ to 18 come

from a floating point computation with a margin far above its rounding error, not from interval arithmetic. The bases of the domination problems come from numerically computed analyses in their sources, used as stated there. Numbers of theorems, lemmas and appendices of [36, 45] refer to their arXiv versions. Code and Lean files are at `https://github.com/ssenge/LAIF`, in the folders `quantum-graph-colouring`, `quantum-dominating-set`, `quantum-monotone-local-search`, `quantum-approximate-monotone-local-search` and `quantum-vertex-cover`, built with Lean v4.36.0-rc1 and Mathlib at revision `d875a8de5222f62cf1c2fafc7984588adadc990a`.

2 Preliminaries

For $\delta \in [0,1]$ let $h(\delta) = -\delta \log \delta - (1-\delta)\log(1-\delta)$, with logarithms to base 2. O^* hides factors polynomial in n.

Backtracking trees. A backtracking tree is given by a root instance and a procedure that, in time $\mathrm{poly}(n)$, decides for an instance whether it is a leaf, and if so whether it is marked, or else lists its children. The depth is at most $\mathrm{poly}(n)$. Given a number T computable in time $\mathrm{poly}(n)$, the quantum walk of [31] decides whether a tree with at most T nodes has a marked leaf, with bounded error, in time $O(\sqrt{T}\,\mathrm{poly}(n))$ and polynomial space. Its circuit depends only on n and T, so it can be used inside quantum search.

Maximal independent sets. By [8], an n-vertex graph has at most $I(n,t) = O^*(2^{E(t/n)n})$ maximal independent sets of size t, for a concave piecewise linear function E with $E(1/s) = (\log s)/s$, and they can be listed by a branching algorithm whose number of leaves is bounded by $I(n,t)$ in the form required by Lemma 4 of [36].

3 3-Colouring and 4-Colouring

Constraint satisfaction. In a $(3,2)$-constraint satisfaction instance, every variable has at most three values and every constraint forbids one pair of values on two variables. A variable with at most two values can be eliminated in polynomial time (Lemma 2 of [5]). The algorithm of Theorem 1 of [5] works by reductions with one call and by branching into a constant number of smaller instances, with polynomial work per call, and it solves its leaves by a bipartite matching. Its analysis bounds the number of recursive calls by $p(r)\,\Lambda^r$ on an instance with r three-valued variables, where $\Lambda < 1.36444$ is the largest root of $x^5 = 2x + 2$. So it is a backtracking tree with at most this number of nodes.

Lemma 3.1. *The 3-colouring algorithm of [5] is a backtracking tree with at most $p(n)\,\beta^n$ nodes on graphs with n vertices, where $\beta = (2^3\,3^4\,\Lambda^{24})^{1/49} < 1.3289$ and p is a fixed polynomial.*

Proof. The algorithm first applies reductions and the branching on cycles of vertices of degree three, each an OR over smaller instances. It then chooses a forest in polynomial time, and tries colourings of a set of vertices, one vertex at a time, with a node that stores the forest, the order and the partial colouring. Each such colouring leaves a $(3,2)$-constraint satisfaction instance. Theorem 2 of [5] bounds the product of these counts with the size of the search trees of the last step, which gives the bound on the nodes, up to the polynomial depth. This is the form used in Lemma 6 of [36]. □

The instances of 4-colouring. We use the framework of [43], with the notation of [45]. An instance is $P = (G, I, S)$, where I is an independent set, $S \supseteq N(I)$ is disjoint from I, the vertices of I get colour 4, the vertices of S get colours in $\{1, 2, 3\}$, and the vertices of $U = V \setminus (I \cup S)$ are free. The instance is terminal if every component of $G[U]$ is a K_3 or a K_4. Let n_3 and n_4 count these components. The measure is

$$\mu(P) = \omega_c|S| + \omega'_3 n_3 + \omega'_4 n_4 + \sum_C \sum_{v \in C} \omega_{\deg_C(v)},$$

summing over the other components C of $G[U]$, with $\omega_0 = 0$ and $\omega_i = 1$ for $i \geq 3$. If the weights satisfy Equation (3) of [43], then $\mu(G, \emptyset, \emptyset) \leq n$.

A nonterminal instance is replaced by the branching rules of Section 3.2 of [43]. Each rule gives instances P_j such that P has a colouring if and only if some P_j has one, and $|U|$ decreases in each P_j. Their number is a constant, except on a cycle component, where the rule takes one P_j for each maximal independent set of the cycle. If the weights and r satisfy Equations (3) and (6) to (21) of [43], then $\sum_j r^{\mu(P_j)} \leq r^{\mu(P)}$, which is Lemma 5.7 of [45]. Every clique of a terminal instance may be assumed to contain a vertex of colour 4, since one of its vertices can be recoloured with colour 4 otherwise: it has no neighbour in I, and its neighbours in S do not use this colour.

Terminal instances. From a terminal instance we compute in polynomial time two maximal families of disjoint copies of $K_{1,3}$, one in $G[S]$ with vertex set S_1, and one with centres in U and leaves in $S \setminus S_1$. Let $s_1 = |S_1|$, let s' be the number of vertices of S in neither family, and let a_i and b_i count the components K_3 and K_4 with exactly i centres. Then $G[S \setminus S_1]$ has maximum degree at most two (Lemma 5.1 of [45]). We use these terminal algorithms, all of them OR trees whose steps take polynomial time and space.

(T1) Choose the vertex of colour 4 in each clique and 3-colour the rest by Lemma 3.1 (Lemma 5.2 of [45]). The size of the tree is $3^{n_3} 4^{n_4} \beta^{|S|+2n_3+3n_4}$ up to polynomial factors.

(T2) Choose the colours of the centres of both families, and restrict the cliques without centres to lists of two colours, then solve the $(3, 2)$-constraint satisfaction instance (Lemma 5.4 of [45]). The size is $3^{s_1/4} \Lambda^{s'} \prod_i g_{3,i}^{a_i} \prod_i g_{4,i}^{b_i} e^{o(n)}$, with $g_{3,0} = 9/2$, $g_{4,0} = 6$, $g_{m,1} = \Lambda^{m-1}(3 + \Lambda^3)$ and $g_{m,i} = \frac{6}{(4-i)!}(4 - i + i\Lambda^3)$ for $i \geq 2$. The factor $9/2$ comes from a family of list assignments on blocks of $O(\log n)$ triangles, chosen greedily in polynomial time (Lemma 5.3 of [45]).

(T3) Enumerate the colours of S_1 and of the cliques, then solve list colouring on the paths and cycles $G[S \setminus S_1]$ in polynomial time (Lemma 5.5 of [45]). The size is $24^{s_1/4} 18^{n_3} 24^{n_4}$ up to polynomial factors.

(T4) Enumerate the sets of at most $n_3/3$ triangles that miss one fixed colour of $\{1, 2, 3\}$, which suffices by the symmetry of these colours, restrict the domains as in Section 3.1 of [43], and solve the $(3, 2)$-constraint satisfaction instances. The size is $1.3645^{|S|} 4.837^{n_3} 6^{n_4}$ up to polynomial factors.

Theorem 3.2. *4-Colouring has a quantum algorithm with bounded error, polynomial space and no quantum random access memory, running in time $O^*(1.6981^{n/2}) \subset O(1.3032^n)$. With the terminal algorithm (T4) alone, the time is $O^*(1.7159^{n/2}) \subset O(1.3100^n)$.*

Proof. The tree starts at $(G, \emptyset, \emptyset)$, branches by the rules above and, at a terminal instance, computes the numbers s_1, s', a_i, b_i and runs the terminal algorithm with the smallest stated size. A node stores the instance and the choices made so far, and a choice among polynomially many values is one step.

On a cycle component the maximal independent set is chosen one vertex at a time, keeping only choices that can still be completed, so each such node lies above some P_j and these nodes add a polynomial factor. The leaves are the leaves of the $(3, 2)$ trees, of the 3-colouring trees, or of the list colouring step, and a leaf is marked if it gives a colouring. So there is a marked leaf if and only if G is 4-colourable, and the depth is polynomial.

For the size, write $x = (s_1, s', a_0, \ldots, a_3, b_0, \ldots, b_4)$, and write each terminal size as $\exp(R_i x)$ with a row R_i read off from (T1), (T2) and (T3). Lemma 5.8 of [45] gives the following. If $\eta \geq 0$ with $\sum_i \eta_i = 1$ and $\ln r\,(\omega_c|S| + \omega'_3 n_3 + \omega'_4 n_4) \geq \sum_i \eta_i R_i x$ for all $x \geq 0$, then $\min_i R_i x \leq \mu(P) \ln r$ at every terminal instance. Since $\sum_L r^{\mu(L)} \leq r^{\mu(G,\emptyset,\emptyset)} \leq r^n$ over the terminal instances L, the number of nodes is at most r^n times a polynomial and a factor $e^{o(n)}$. All constraints hold at $r = 1.69804$ with

$$\omega_1 = 0.9342533916, \quad \omega_2 = 1, \quad \omega_c = 0.5593776725, \quad \omega'_3 = 2.9704494860, \quad \omega'_4 = 3.5579545637,$$

and $\eta = (0.0822508347, 0.8778792406, 0.0398699247)$. Our script checks each constraint with 60 digits and a margin of 10^{-9}. So there are constants C and a with at most $T_n = C\, n^a\, 1.6981^n$ nodes on every graph with n vertices. The walk of [31] with this bound decides 4-colourability in time $O^*(1.6981^{n/2})$, the same for every input of size n, and $1.6981 \leq 1.3032^2$. For (T4) alone, the parameters of Table 2 of [43] satisfy its Equations (3) to (21), where (4) and (5) bound the size of (T4) by $r^{\mu(P)}$ at $r = 1.7159$. This is the proof of Theorem 8 of [43], and it gives the bound T_n with base 1.7159, and $1.7159 \leq 1.3100^2$.

A colouring follows by self-reduction. Add a K_4 on four new vertices. For each vertex v in turn, merge v with one of the new vertices, and keep the first merge that leaves a 4-colourable graph. The graph has at most $n + 4$ vertices throughout, so this costs a factor $O(n)$ and a constant factor in the time. □

The classical bound $O^*(1.6975^n)$ of [45] uses its 3-colouring algorithm in (T1), which needs exponential space, while its (T2) and (T3) do not. So the classical form of the tree above is a polynomial-space 4-colouring algorithm in time $O(1.6981^n)$, implied by [43, 45].

Remark 3.3. The work [29] gives a 3-colouring algorithm through a new choice of the forests and a linear program over their structure. The running time stated there is $O^*(1.3217^n)$, while the program as printed has optimum 1.32702 [45, Appendix F]. This does not bound the running time from below, and further constraints on the forests may lower the optimum. The algorithm has the steps of Lemma 3.1. Its vertices are coloured in the order of its forests, so it is a backtracking tree. If the constraints of that program hold for its forests, it has at most $p(n)\, 1.32703^n$ nodes. In that case the same computation gives quantum time $O(1.1520^n)$ for 3-colouring, $O(1.5255^n)$ for $k = 6$, $O(1.8242^n)$ for $k = 12$ and $O(1.9574^n)$ for $k = 19$, and $O(1.7722^n)$ for $k = 11$, with the other rows of Table 1 unchanged. If 1.3217 holds, the 3-colouring bound becomes $O(1.1497^n)$.

4 k-Colouring

4.1 The recursion

Let $\textsc{Col}(G, k, \mu)$ be a procedure that answers yes if the vertex set of G, of size n, splits into k independent sets of size at most μn each, answers no if G is not k-colourable, and may answer either otherwise. Then $\textsc{Col}(G, k, 1)$ decides k-colourability. Calling Col with a larger μ only weakens the first condition, so it is always correct to replace μ by a larger value.

Theorem 9 of [36] gives such a procedure for $k \geq 4$ from two reductions, each with quantum search over its choices.

- **Reduction 1.** Search over the maximal independent sets I of size $t \geq n/k$, and call $\textsc{Col}(G - I, k-1, 1)$. In a colouring with k classes the largest class can be extended to a maximal independent set, which has size at least n/k.
- **Reduction 2.** Fix $2 \leq k' \leq k/2$. Search over the sets S of size t with $k'n/k \leq t \leq \min(n, \mu k' n)$, and answer yes if $\textsc{Col}(G[S], k', \mu n/t)$ and $\textsc{Col}(G - S, k - k', \lfloor t/k' \rfloor/(n-t))$ both answer yes. For $t = n$ the graph $G - S$ is empty and the second call is omitted. If S is the union of the k' largest classes of a colouring with classes of size at most μn, then S has this size, its classes have size at most μn, and the remaining $k - k'$ classes are at most the average t/k' of the k' largest ones.

Let f_3 and f_4 be exponents of 3-colouring and 4-colouring, let $d_3 = f_3$ and $d_1 = d_2 = 0$, and for $k \geq 4$ and $\mu \in [1/k, 1]$ let

$$d_k(\mu) = \min\Big(\max_{1 \leq s \leq k} \frac{\log s}{2s} + \Big(1 - \frac{1}{s}\Big) d_{k-1}(1), \tag{1}$$

$$\min_{2 \leq k' \leq k/2} \max_{\delta \in [k'/k,\, \min(1, \mu k')]} \frac{h(\delta)}{2} + \max\Big(\delta\, d_{k'}\big(\min(1, \tfrac{\mu}{\delta})\big), \tag{2}$$

$$(1-\delta)\, d_{k-k'}\big(\min(1, \tfrac{\delta}{k'(1-\delta)})\big)\Big)\Big).$$

Line (1) is the time of Reduction 1: quantum search over $I(n,t)$ sets costs $2^{E(\delta)n/2}$ for $\delta = t/n$, the maximum of $E(\delta)/2 + (1-\delta)d_{k-1}(1)$ over $\delta \in [1/k, 1]$ is attained at some $\delta = 1/s$, since E is piecewise linear with breakpoints at these points (Corollary 3 of [36]), and $E(1/s) = (\log s)/s$. The term $s = 1$ is 0 and is included only to make the range simple; [36] uses $2 \leq s \leq k$. Line (2) is the time of Reduction 2, with $\binom{n}{t} = O^*(2^{h(\delta)n})$ sets. By Theorem 9 of [36], k-colouring has a quantum algorithm in time $O^*(2^{d_k(1)n})$ with bounded error, polynomial space and no quantum random access memory. An algorithm that decides 4-colourability is a correct $\textsc{Col}(\cdot, 4, \mu)$ for every μ, so the bound f_4 may replace $d_4(\mu)$ when it is smaller. The depth of the recursion is at most $k \leq 20$, so the factors from nested search and error reduction stay polynomial.

4.2 Evaluation as an upper bound

The maximum over δ in (2) has no closed form, and $d_{k'}$ and $d_{k-k'}$ are needed at many arguments. We replace both by quantities that the algorithm itself can use.

Fix a grid $\mu_j = j/M$ for $j = 0, \ldots, M$, and let $\lceil \mu \rceil_M$ be the smallest grid point at least μ, for $\mu \leq 1$. For each range $[k'/k, \min(1, \mu_j k')]$, fix a partition into P pieces $[a, b]$. Define $\hat{d}_1 = \hat{d}_2 = 0$, $\hat{d}_3 = f_3$, and for $k \geq 4$ let $\hat{d}_k(\mu_j)$ be the minimum of f_4 if $k = 4$, of line (1) with $\hat{d}_{k-1}(1)$ in place of $d_{k-1}(1)$ and of

$$\min_{2 \leq k' \leq k/2} \max_{[a,b]} \frac{\max_{\delta \in [a,b]} h(\delta)}{2} + \max\Big(b\, \hat{d}_{k'}\big(\lceil \min(1, \tfrac{\mu_j}{a}) \rceil_M\big),\ (1-a)\, \hat{d}_{k-k'}\big(\lceil \min(1, \tfrac{b}{k'(1-b)}) \rceil_M\big)\Big), \tag{3}$$

where the argument of $\hat{d}_{k-k'}$ is 1 when $b = 1$.

Proposition 4.1. *For every $k \geq 1$ and grid point $\mu_j \geq 1/k$, there is a quantum procedure $\textsc{Col}(\cdot, k, \mu_j)$ with bounded error, polynomial space and no quantum random access memory, in time $O^*(2^{\hat{d}_k(\mu_j)n})$.*

Proof. By induction on k. For $k \leq 2$ colouring takes polynomial time, and for $k = 3$ we use Lemma 3.1 and the walk of [31] and ignore μ. For $k = 4$ we may also use Theorem 3.2 and ignore

μ. Let $k \geq 4$. Reduction 1 calls $\textsc{Col}(\cdot, k-1, 1)$, and $1 = \mu_M$ is a grid point, so its time is line (1) with $\hat{d}_{k-1}(1)$.

For Reduction 2 with a fixed k', every size t in the search has $\delta = t/n$ in some piece $[a, b]$. For t in this piece we call

$$\textsc{Col}\big(G[S], k', \lceil \min(1, \mu_j/a) \rceil_M\big) \quad \text{and} \quad \textsc{Col}\big(G - S, k - k', \lceil \min(1, b/(k'(1-b))) \rceil_M\big).$$

These limits are at least the ones of Reduction 2: since $t \geq an$, the classes of $G[S]$ have size at most $\mu_j n \leq (\mu_j/a)\, t$, and since $\delta \leq b$, the remaining classes have size at most $t/k' \leq (b/(k'(1-b)))(n-t)$. A limit of 1 always holds, and it is the one used when $b = 1$, where $G - S$ can still have up to $(1-a)n$ vertices. So the procedure stays correct. These limits lie in the domain of $\hat{d}$: since $a \leq \mu_j k'$ we have $\mu_j/a \geq 1/k'$, and since $b \geq k'/k$ we have $b/(k'(1-b)) \geq 1/(k-k')$. For fixed k, M and P there are finitely many pieces and grid points, so all polynomial factors are uniform. By induction the two calls take time $O^*(2^{\hat{d}_{k'}(\cdot)\, t})$ and $O^*(2^{\hat{d}_{k-k'}(\cdot)(n-t)})$ at the stated grid points, and $t \leq bn$ and $n - t \leq (1-a)n$. Quantum search over the $\binom{n}{t} = O^*(2^{h(\delta)n})$ sets S of size t costs a factor $2^{h(\delta)n/2} \leq 2^{n \max_{[a,b]} h/2}$. Summing over the polynomially many t and taking the best k' gives (3). The procedure is run with k and the reduction fixed in advance, and the base cases have a running time that depends only on n, so the circuit is the same for every input of size n. □

A call $\textsc{Col}(G, k, 1)$ decides k-colourability, so $\hat{d}_k(1)$ is an exponent for k-colouring.

4.3 The values

Take $f_3 = \frac{1}{2} \log \beta$ from Lemma 3.1, and $f_4 = \frac{1}{2} \log 1.6981$ from Theorem 3.2, or $f_4 = \frac{1}{2} \log 1.7159$ for the column with [43] alone.

Rows $k = 4, 5$. Theorem 3.2 gives the row $k = 4$. For $k = 5$ line (1) suffices. Its term for s equals log of $s^{1/(2s)} x^{(s-1)/s}$ with $x = 2^{d_4(1)}$, and this is at most $\log y$ whenever $s\, x^{2(s-1)} \leq y^{2s}$. With $x = \sqrt{1.6981}$ and $y = 1.4518$, and with $x = \sqrt{1.7159}$ and $y = 1.4578$, these are polynomial inequalities for $s = 1, \ldots, 5$, all true.

Rows $k = 6$ to 20. We evaluate (3) with $M = P = 4000$, equal pieces, and add 10^{-9} to each exponent, which is far above the floating point error. Table 1 shows the results, rounded up. The optimal k' at $\mu = 1$ is $\lfloor k/2 \rfloor$ for $k = 6, \ldots, 12$, and $6, 6, 7, 7, 7, 8, 8, 8$ for $k = 13, \ldots, 20$.

As a test of the evaluation, the same computation without f_4 gives the precise values of Tables 2 and 3 of [36] to within 0.0008 in the base, and in all rows at least these values, as expected from an upper bound.

5 Domination problems

A dominating set of a graph is a set D of vertices such that every vertex is in D or has a neighbour in D. An independent dominating set is a dominating set with no edge inside it, and an edge dominating set is a set of edges such that every edge shares an endpoint with one of them. Minimum Maximal Matching asks for a smallest maximal matching. In each case the decision version asks for a solution of size at most k.

Theorem 5.1. *With bounded error, polynomial space and no quantum random access memory, Dominating Set has a quantum algorithm in time $O^*(1.4969^{n/2}) \subset O(1.2235^n)$, Independent Dominating Set one in time $O(1.1583^n)$, and Edge Dominating Set, Minimum Maximal Matching and*

Problem	Previous best bound	This paper
Dominating Set	$2^{n/2}$ quantum [42]; 1.3100^n quantum from [18]; 1.4864^n [24], 1.4969^n [33] classical	$\mathbf{1.2235^n}$
Independent Dominating Set	1.3351^n [6], $2^{0.424n}$ [7] classical; 1.2010^n quantum	$\mathbf{1.1583^n}$
Edge Dominating Set	1.3160^n [44], 1.3226^n [34] classical; 1.2010^n quantum	$\mathbf{1.1501^n}$
Minimum Maximal Matching	1.3160^n classical [44, 34]; 1.2010^n quantum	$\mathbf{1.1501^n}$
Weighted versions of the last two	1.3226^n classical [34]; 1.2010^n quantum	$\mathbf{1.1501^n}$

Table 2: Domination problems on n vertices. The classical bounds of [24, 33, 34, 7] are for polynomial space, and the previous bounds are the best known to us. The quantum bounds are for bounded error, polynomial space and no quantum random access memory, up to polynomial factors. The quantum value 1.2010^n is the walk on the branching enumeration of maximal independent sets, with at most $3^{n/3}$ leaves [30], and for edge domination of the minimal vertex covers; to our knowledge it has not been stated. The value 1.3100^n is the walk on the enumeration of minimal dominating sets, if that enumeration is a backtracking tree. The bound 1.3160^n for Minimum Maximal Matching is implied by [44] through the conversion in [34]. For unweighted graphs, Minimum Maximal Matching and Edge Dominating Set have the same optimum, with a conversion in polynomial time [34].

their weighted versions with non-negative integer weights of polynomial bit length have quantum algorithms in time $O^*(1.3226^{n/2}) \subset O(1.1501^n)$.

Proof. For each problem we show that the decision version of the classical algorithm, with a budget k, is a backtracking tree. Each step of these algorithms has the form $\mathrm{opt}(x) = \min_i(c_i + \mathrm{opt}(x_i))$ with values c_i computed in polynomial time, which is equivalent to the OR of the decisions on x_i with budget $k - c_i$. So each node answers, makes one call with a new budget, or branches into the OR of polynomially many children; it computes them in polynomial time and stores an instance of polynomial size, and the depth is polynomial. A node is labelled by its path of child indices in binary, so that the walk sees a tree of constant degree, also where two paths reach the same instance. The classical analysis bounds the number of leaves of the full search tree by $p(n)\,c^n$ for the classical base c and a fixed polynomial p, and the tree for a budget k is a subtree of it. The depth is at most $2n$ for Dominating Set, since each step removes a set or an element and the rule below removes three sets and two elements and adds one set; at most n for edge domination, since each step lowers the number of undecided vertices; and at most $2n + n^2$ for Independent Dominating Set, counting vertices, free vertices and edges. So the tree has at most $T_n = (2n + n^2 + 1)\,p(n)\,c^n$ nodes. The walk of [31] with the bound T_n then decides the problem in time $O^*(c^{n/2})$, the same for every input of size n. Trying every budget, or a binary search for integer weights, gives the optimum. A solution follows by descending the tree from the root, running the detection on each child and moving to one that contains a marked leaf, then replaying the path and undoing each reduction as in the sources, in polynomial time; with errors amplified to $1/\mathrm{poly}(n)$ per call this costs a polynomial factor.

Dominating Set. Algorithm 4 of [33], in the numbering of its technical report, works on the set cover instance with the closed neighbourhoods as sets over the vertices. If no element is left, it answers yes when $k \geq 0$; if $k \leq 0$ or an element lies in no set, it answers no; if all sets have at most two elements, a maximum matching gives the minimum cover, which it compares with k. Its

reduction rules make one call each. An element of frequency one forces its set, with budget $k-1$. A set contained in another set, and an element whose sets include all sets of another element, are removed, with budget k. The counting rule of [33] takes a set, with budget $k-1$. The rule for a set $R=\{e_1,e_2\}$ of two elements of frequency two, with second sets R_1 and R_2, replaces R,R_1,R_2 by $(R_1\cup R_2)\setminus R$ on the elements outside R. Since the subset rules are applied first, $R_1\neq R_2$ and $R\not\subseteq R_i$. By the argument in [33], the minimum cover then drops by exactly one, so the call has budget $k-1$. Otherwise the algorithm branches on a set S of maximum size, into taking S with budget $k-1$ or discarding S with budget k. Its running time is $O^*(1.4969^n)$ in polynomial space [33], and $\sqrt{1.4969}\leq 1.2235$.

Independent Dominating Set. The algorithm of [7] keeps vertices free or marked; a marked vertex may not enter the solution and must still be dominated. For a chosen vertex v, some free $u\in N[v]$ is in the solution, and the algorithm branches over these u, removes $N[u]$ and lowers the budget by one, marking the vertices tried in earlier branches. Its other steps have the same form: binary branches that take a vertex or mark it, branches that take several vertices together, forced additions, removal of equivalent vertices and of edges between marked vertices, and prunings of branches that cannot improve the solution. The analysis uses a measure of at most n that is a sum of non-negative vertex weights. Some branchings are analysed there together with the branching that follows on a marked vertex of degree at most two created by the first one. We let the algorithm branch next on that vertex and count the two steps as one node. With this, the cases of the analysis give factor at most $1.3413<c_0=2^{0.424}<1.34165$ at every node other than a branching on a vertex of minimum degree in a regular component with degree $\delta\in\{2,3,4,5\}$, which we call bad. The recurrence printed in the statement of Lemma 6 of [7] has factor 1.4051, while each case in its proof has factor at most 1.3413. A vertex of minimum degree with a neighbour of larger degree is preferred, so all minimum-degree vertices lie in regular components at a bad branching. Remark 1 of [7] counts bad branchings as finitely many. On a disjoint union of regular components they can occur once per component, so we complete the argument with one more step. Fix $K=10^7$. A connected component H with at most K vertices is solved directly in constant time, and the algorithm makes one call on the rest with budget $k-\mathrm{opt}(H)$, or answers no if H has no solution. If this step removes the component of the marked vertex between the two steps of a combined node, that branch drops by the whole measure of the component, which is at least the drop in each child of the second step, since those children change only vertices of the component. So the combined node keeps its bound. Now take two bad branchings on a path from the root, with components H and later H'. No vertices are added and degrees do not increase, and the first branching removes a vertex of H. So either H' is disjoint from H, or H' is a proper subset of the connected graph H, in which case it has smaller degree. These components therefore form a laminar family of depth at most 4. Each of them has more than K vertices, since smaller components are removed first, and the members at the same height in the family are disjoint. So a path has at most $4n/K$ bad branchings. A bad branching has at most $\delta+1\leq 6$ children and the measure does not increase, so its sum $\sum_i c_0^{-\mathrm{drop}_i}$ is at most 6, and every other node, combined as above, has sum at most 1. Giving each leaf the product along its path of $c_0^{-\mathrm{drop}_i}$ divided by the sum at the node, the weights of the leaves add up to one, and each weight is at least $c_0^{-n}\,6^{-4n/K}$. So there are at most $(c_0\,6^{4/K})^n$ leaves, and $1.34165\cdot 6^{1/2500000}\leq 1.1583^2$. The algorithm with this extra step uses polynomial space.

Edge Dominating Set and Minimum Maximal Matching. Algorithm 4 of [34], in the numbering of its technical report, splits the vertices into a vertex cover C, an independent set I and undecided vertices U. A clique component H of $G[U]$ of any size is handled by one call on the graph with a new vertex v joined to H, with $C\cup H\cup\{v\}$ and $U\setminus H$, whose optimum is exactly one larger, so the call has budget $k+1$. The new vertices keep the graph at most $2n$ vertices. Otherwise it

branches on an undecided vertex, into I with its undecided neighbours into C, or into C, and on the paths and cycles of $G[U]$ into two or a constant number of children. At a leaf C is a vertex cover, and the smallest edge dominating set whose endpoints include C is computed through a maximum matching. Unweighted Minimum Maximal Matching has the same decision problem by the conversion in [34]. For the weighted versions, Algorithm 5 of [34] replaces the clique rule by its Rule 2, which puts isolated undecided vertices into I with budget k, and its Rule 3, one call on cliques of two or three vertices with the budget raised by the weight of the new edge, and larger cliques are branched on. Algorithm 7 of [34] replaces the clique rule by its Rule 4, one call on a clique component of any size with zero-weight new edges and budget k. Their leaves are a generalized edge cover or a perfect matching, both computed in polynomial time. The running time is $O^*(1.3226^n)$ in polynomial space, and $\sqrt{1.3226} \le 1.1501$. □

The faster algorithms of [24] for Dominating Set, of [6] for Independent Dominating Set and of [44] for Edge Dominating Set and unweighted Minimum Maximal Matching would give the bounds $O(1.2192^n)$, $O(1.1555^n)$ and $O(1.1472^n)$ by the same argument if they are backtracking trees in this sense. Their structure is not covered here.

6 Monotone local search

Monotone local search [17] turns an algorithm that extends a partial solution into an exact algorithm. An implicit set system Φ maps an instance I of size $N \ge n$ to a universe U_I of n elements and a family F_I of subsets, with membership decidable in polynomial time. The problem Φ-Subset asks for a set in F_I. The problem Φ-Extension asks, for $X \subseteq U_I$ and an integer k, whether some $S \subseteq U_I \setminus X$ with $|S| \le k$ has $X \cup S \in F_I$. If Φ-Extension is solved in time $c^k N^{O(1)}$, then Φ-Subset is solved by a randomized algorithm in time $(2-1/c)^n N^{O(1)}$ [17].

Theorem 6.1. *Let $c > 1$, and suppose that Φ-Extension is decided by a backtracking tree with at most $p(N)\,c^k$ nodes, for a fixed polynomial p. Then Φ-Subset has a quantum algorithm with bounded error, polynomial space and no quantum random access memory, in time $(2-1/c)^{n/2} N^{O(1)}$. The same holds in the permissive form where the tree, on (I, X, k), accepts whenever some S with $|S| \le k$ has $X \cup S \in F_I$, and each accepting leaf gives a set S of any size with $X \cup S \in F_I$.*

Proof. Fix $k' \le n$, and let $t \le k'$ minimize $\binom{n}{t}\binom{k'}{t}^{-1} c^{k'-t}$. Prepare the uniform superposition over the subsets $Y \subseteq U_I$ of size t, which takes polynomially many gates. On each Y, run the walk of [31] on the tree of $(I, Y, k'-t)$ with the bound $T = p(N)\,c^{k'-t}$, with its error reduced to δ by a majority over $O(\log 1/\delta)$ runs and with measurements deferred: compute the answer, copy it into one qubit, and uncompute. This oracle takes time $\sqrt{T}\,\mathrm{poly}(N)$, the same for every Y. Let $p_0 = \binom{k'}{t}\binom{n}{t}^{-1}$ and $m = \lceil p_0^{-1/2} \rceil$. Measure once without amplification and once after j rounds of amplitude amplification [3, 4], for j uniform in $\{0, \ldots, m-1\}$, and accept if a measured Y passes the extension decision again, with a majority over $O(\log n)$ runs. Do this for every $k' \le n$. The walk needs an explicit bound T on the nodes of the full tree, explored without stopping at an accepting leaf.

If F_I contains a set S of size k', every $Y \subseteq S$ with $|Y| = t$ is a yes-instance of the extension, with $S \setminus Y$ as completion, so with an exact oracle the success probability $p = \sin^2\theta$ of the start state is at least p_0. If $p > 3/4$, the measurement without amplification succeeds with probability above $3/4$. Otherwise $\sin 2\theta = 2\sqrt{p(1-p)} \ge \sqrt{p} \ge \sqrt{p_0}$, so $m \sin 2\theta \ge 1$. Then

$$\frac{1}{m} \sum_{j<m} \sin^2((2j+1)\theta) \;=\; \frac{1}{2} - \frac{\sin(4m\theta)}{4m \sin 2\theta} \;\ge\; \frac{1}{4}.$$

An oracle with error δ differs from the exact one by $O(\sqrt{\delta})$ in norm per call, so $\delta = 1/(m^2\mathrm{poly}(n))$, at a cost of $O(\log m) = O(n)$ repetitions, changes this by at most $1/100$. If F_I is empty, no sample is marked, also in the permissive form, and the final check rejects except with small probability. A constant number of repetitions gives bounded error.

The run for k' takes time $m\sqrt{T}\,\mathrm{poly}(N) = (\binom{n}{t}\binom{k'}{t}^{-1}c^{k'-t})^{1/2}\mathrm{poly}(N)$, the square root of the classical run in [17]. By Lemma 2.2 of [17], $\max_{k'}\min_t \binom{n}{t}\binom{k'}{t}^{-1}c^{k'-t} \le (2-1/c)^n n^{O(1)}$, so the total time over all $k' \le n$ is $(2-1/c)^{n/2}N^{O(1)}$. The space is that of the walk and of n sample qubits. A solution follows from an accepted Y by descending the extension tree with the walk at each child and reading it off the accepting leaf. In the exact form, if the leaves do not give S, use self-reduction: stop if the current set is in F_I, and otherwise for v outside the current set Y, run the decision on $(I, Y \cup \{v\}, b-1)$ with the remaining budget b and keep the first v accepted. □

Problem	Extension algorithm	c	Classical, same extension	This paper
Feedback Vertex Set	branching [21]	3.460	1.7110^n	$\mathbf{1.3081^n}$
Cluster Vertex Deletion	branching [38]	1.811	1.4479^n	$\mathbf{1.2033^n}$
Subset and Group Feedback Vertex Set	LP-branching [22]	4	1.7500^n	$\mathbf{1.3229^n}$
Node Unique Label Cover, fixed labels Σ	LP-branching [22]	$\lvert\Sigma\rvert^2$	$(2-\lvert\Sigma\rvert^{-2})^n$	$\mathbf{(2-\lvert\Sigma\rvert^{-2})^{n/2}}$
3-Hitting Set, also weighted	branching [37]	2.168	1.5388^n	$\mathbf{1.2405^n}$
Proper Interval Vertex Deletion	branching [9]	6	1.8334^n	$\mathbf{1.3541^n}$
Multicut on Trees	branch and search [25]	$\sqrt{\sqrt{2}+1}$	1.3565^n	$\mathbf{1.1647^n}$
Min-Ones d-SAT, $d \ge 3$; d-Hitting Set, $d \ge 4$	clause branching [17]	d	$(2-1/d)^n$	$\mathbf{(2-1/d)^{n/2}}$

Table 3: Corollaries of Theorem 6.1. For Multicut on Trees, n is the number of edges. For $d = 3, 4, 5$ the last row gives 1.2910^n, 1.3229^n and 1.3417^n. The classical column applies [17] to the same extension algorithm; for d-Hitting Set [17] gives faster classical bounds through other extension algorithms, such as 1.5182^n for 3-Hitting Set.

Theorem 6.2. *With bounded error, polynomial space and no quantum random access memory, the problems of Table 3 have quantum algorithms with the running times listed there, up to polynomial factors.*

Proof. In each case, removing X (deleting the vertices, cutting the edges or setting the variables to one and simplifying; for hitting sets, removing the hyperedges hit by X and lowering the weight bound by the weight of X) turns the extension instance (I, X, k) into an instance of the same problem with budget k, and we check that the algorithm is a backtracking tree whose number of nodes is bounded by its stated running time. The measure and conquer analyses prove a bound Cc^k on the leaves of the full tree by induction on the measure, for a constant C that is hard-wired into the bound, and the depth is a fixed polynomial, which gives the explicit bound T.

Feedback Vertex Set. The algorithm of [21] works on (G, F, k) with undeletable vertices F, empty at the start. Its reductions delete a vertex, bypass a vertex of degree two or lower a multiplicity, each with one call of budget k or $k-1$. A reduction to matroid matching answers in polynomial time

when the maximum degree is at most three, and a pruning rule answers no. Otherwise it branches into deleting a vertex u of maximum degree, with budget $k-1$, or contracting u with its neighbours in F into F, with budget k. Every branch removes a free vertex and every reduction shrinks the multigraph, so the depth is polynomial. The measure of [21] is k at the start and nonnegative after pruning, so the full tree has $O^*(3.460^k)$ leaves, and $\sqrt{2-1/3.460} \leq 1.3081$.

Cluster Vertex Deletion. The reductions of [38] answer or make one call. Its branching rules branch over the twins of a vertex v and over the members of a dominating family of vertex covers of an auxiliary graph, computed by a recursion that branches on a constant number of sets in each step. This family can be large, so we expand the recursion inside the tree: each of its leaves X is followed by the call on $G - X$ with budget $k - |X|$, and the same is done for the second family built by one of the rules. Section 6 of [38] counts the calls of both the main recursion and the inner recursion, including inner lists with arbitrary covers, and bounds them by $O^*(1.811^k)$, and $\sqrt{2-1/1.811} \leq 1.2033$.

Group and Subset Feedback Vertex Set, Node Unique Label Cover. The algorithms of [22] are LP-branching. Each node solves a half-integral relaxation in polynomial time and fixes, by single calls, assignments that do not raise the relaxed optimum. Otherwise it branches into two children for Group Feedback Vertex Set (Lemma 8 of [22]) or $|\Sigma|$ children for Unique Label Cover (Lemma 1 and Corollary 4 of [22]), and each branch lowers the gap between the budget and the relaxed optimum by at least $1/2$. So there are at most 4^k or $|\Sigma|^{2k}$ leaves. Subset Feedback Vertex Set is equivalent to its edge version, with the edges at the terminals as the edge subset, on the same graph and with the same budget [12], and the edge version reduces to Group Feedback Vertex Set on the same graph with the same budget by Lemma 5 of [11], with the group $\mathbb{Z}_2^{|S|}$ for the edge subset S. Since this group is large, the relaxation is solved in the oracle model of Section 5.2 of [22]. For Group Feedback Vertex Set $\sqrt{2-1/4} \leq 1.3229$; for Unique Label Cover $|\Sigma|$ is fixed.

3-Hitting Set and Weighted 3-Hitting Set. The algorithm of Appendix B.1 of the arXiv version of [37], for positive integer weights of polynomial bit length, works on 3-uniform hypergraphs with a weight bound W and a size parameter k, and some of its rules create smaller hyperedges inside a step. Its reductions answer, solving hypergraphs of degree at most two in polynomial time, or make one call, one of them with modified weights, and each branching rule returns the first of two answers that is not NIL. A non-NIL answer is a hitting set of weight at most W, and one is returned whenever a hitting set of weight at most W and size at most k exists, which is the permissive form of Theorem 6.1, and the answer gives the set. Its measure $k-\alpha(G)+1$ with $\alpha \leq 0.87$ and branching numbers at most 2.168 give $O^*(2.168^k)$ leaves. The family F_I is the set of hitting sets within the weight bound. Unit weights give 3-Hitting Set, and $\sqrt{2-1/2.168} \leq 1.2405$.

Proper Interval Vertex Deletion. If G has an induced subgraph from a finite family of graphs with at most six vertices, found in linear time, the algorithm of [9] branches into deleting each of its vertices with budget $k-1$; otherwise the problem is solved in linear time. So there are at most 6^k leaves, and $\sqrt{2-1/6} \leq 1.3541$. Interval Vertex Deletion is not included, since the algorithm of [10] solves components and modules to optimality before it continues.

Multicut on Trees. Cutting X removes its edges and the requests they separate and leaves a forest, which the algorithm of [25] handles, since it cuts edges itself. Its reductions are single calls, and those that use minimum vertex covers do so on auxiliary graphs of maximum degree two, in polynomial time. The branching rules make two or a constant number of children, the depth is polynomial, and the number of leaves is at most ρ^k with $\rho = \sqrt{\sqrt{2}+1}$. Then $\sqrt{2-1/\rho} \leq 1.1647$.

Min-Ones d-SAT and d-Hitting Set. The algorithm of [17] picks an unsatisfied clause and branches over its at most d variables set to one, with budget $k-1$, so there are at most d^{k+1} nodes. □

Quantum search over all subsets takes time $2^{n/2}$. For Feedback Vertex Set, the walk on the algorithm of [32] gives $O(1.3748^n)$, as stated for Maximum Induced Forest in [28]. The walk on a classical exact algorithm with base b gives $\sqrt{b}$ when that algorithm is a backtracking tree. The exact bases listed in [17] give at best $\sqrt{1.7347} > 1.3170$ for Feedback Vertex Set, $\sqrt{1.6181} > 1.2720$ for Cluster Vertex Deletion, $\sqrt{1.8638} > 1.3652$ for Subset Feedback Vertex Set and $\sqrt{1.8704} > 1.3676$ for 4-Hitting Set, and [17] lists no exact algorithm faster than brute force for Group Feedback Vertex Set, Node Unique Label Cover, Multicut on Trees and Min-Ones d-SAT, and for Proper Interval Vertex Deletion only $(2-\varepsilon)^n$ with $\varepsilon < 10^{-20}$. To our knowledge, no quantum form of monotone local search has been stated before. The classical column of Table 3 uses the same extension algorithms; faster classical bounds exist for some rows, such as 1.6097^n for Min-Ones 3-SAT [17]. For 3-Hitting Set the walk on the exact algorithm of [40] would give $\sqrt{1.6278} > 1.2758$ if it is a backtracking tree, and the 2.0755^k extension algorithm of [40] would give 1.2322^n through Theorem 6.1 if it is one. The faster 2.0409^k algorithm of [39] computes the optimum of a component by recursive calls before a tail call and is not a plain OR tree, so it is not used. Problems that reduce to 3-Hitting Set on the same universe, such as Feedback Vertex Set in Tournaments, get the bound 1.2405^n as well, although their own extension algorithms may give better bounds if they are backtracking trees.

6.1 Approximation

Let F_I be nonempty and upward closed, let OPT be the smallest size of a set in F_I, and call a set in F_I of size at most $\beta\,\mathrm{OPT}$ a β-approximation. Approximate monotone local search [13, 14] turns extension algorithms with ratio α and running time c^k into exponential-time β-approximations. Let L be a finite list of pairs (α, c) with $\alpha, c \geq 1$ and let $\beta \geq 1$. For a guess k of OPT, a pair (α, c) and $t \leq \beta k$, let $x = \max(0, \lceil (1-\beta/\alpha)k + t/\alpha \rceil)$ and $\ell = k - x$, and let $h(n, k, t, x)$ be the probability that a uniform t-subset of $[n]$ meets $[k]$ in at least x elements. Let $\mathrm{best}(L, \beta)$ be the base of $\max_{k \leq n/\beta} \min_{(\alpha,c),t} c^\ell / h(n, k, t, x)$, up to polynomial factors. This is a variant of the bound in Equations (5) and (6) and Theorem 2.3 of [14], which agrees with it up to a constant factor in the cost.

Theorem 6.3. *Let $\beta \geq 1$, and let each pair $(\alpha, c) \in L$ come with an extension procedure of one of two kinds: (a) a randomized polynomial-time algorithm with branching probabilities at least $1/\mathrm{poly}(N)$ and polynomial depth that, on (I, X, ℓ) such that some S with $|S| \leq \ell$ has $X \cup S \in F_I$, returns Y with $X \cup Y \in F_I$ and $|Y| \leq \alpha\ell$ with probability at least $q(N, \ell) \geq c^{-\ell}/p(N)$, where q is computable in polynomial time and the bound still holds after the probabilities are rounded to $O(\log N)$ bits, at the loss of a constant factor; or (b) for $\alpha = 1$, a backtracking tree with at most $p(N)c^\ell$ nodes that accepts (I, X, ℓ) exactly when such an S exists, and whose accepting leaves give one. Then there is a quantum β-approximation for Φ with bounded error, polynomial space and no quantum random access memory, in time $\mathrm{best}(L, \beta)^{n/2} N^{O(1)}$.*

Proof. For each $k \leq n/\beta$, take the pair and t that minimize $c^\ell / h(n, k, t, x)$. Prepare the uniform superposition over the t-subsets X, together with the random bits of the procedure for kind (a), with its branching probabilities rounded. Mark a branch if $Z = X \cup Y \in F_I$ and $|Z| \leq \beta k$ for kind (a), or if the detection walk accepts (I, X, ℓ) for kind (b), with its error reduced as in the proof of Theorem 6.1. If $k = \mathrm{OPT}$ and $|X \cap \mathrm{OPT}| \geq x$, the rest of the optimum completes X within ℓ, and $|Z| \leq t + \alpha\ell \leq \beta k$, since $\alpha x \geq (\alpha - \beta)k + t$, or $x = 0$ and $t + \alpha k \leq \beta k$. So the marked probability is at least $q_0 = h(n, k, t, x)$ times $q(N, \ell)$ for kind (a). With $m = \lceil 1/\sqrt{q_0} \rceil$, one measurement without amplification and one after a uniform number of rounds below m succeed with probability at least $1/4$, as in the proof of Theorem 6.1. For kind (b), Y is found by descending the tree. The run costs $\sqrt{c^\ell / h(n, k, t, x)}\,\mathrm{poly}(N)$, the square root of the classical cost, and the smallest checked set

over all k is returned, together with U_I as a candidate, which lies in F_I by monotonicity and is a β-approximation if $\mathrm{OPT} > n/\beta$. □

The randomized branching algorithms of [26] for Vertex Cover and 3-Hitting Set are polynomial-time procedures of kind (a) that succeed with probability at least $(c^*+\varepsilon)^{-k}$ for every $\varepsilon > 0$ and large k, where the numerically evaluated bases c^* are rounded up to the tabulated c. If the hypothesis holds only in this form, the proof gives $(\mathrm{best}(L,\beta)+\varepsilon)^{n/2}$ for every $\varepsilon > 0$, and since best is continuous in the bases, the rounded-up entries below still hold. Polynomial-time approximations are of kind (a) with $c = 1$: a maximal matching for Vertex Cover, a maximal family of disjoint hyperedges for d-Hitting Set, and the algorithms of [2, 16, 1, 41, 23] for Feedback Vertex Set, Subset Feedback Vertex Set, Cluster Vertex Deletion, Proper Interval Vertex Deletion and Multicut on Trees, with ratios $2, 8, 2, 6, 2$. The trees of Theorem 6.2 are of kind (b), since they count their budget and accept only within it; for 3-Hitting Set we use unit weights and $W = \ell$, so that an answer other than NIL has size at most ℓ. Table 4 gives the resulting bounds.

Problem	$\beta = 1.1$	$\beta = 1.2$	$\beta = 1.5$	$\beta = 1.8$	exact, quantum
Vertex Cover	$\mathbf{1.0555^n}$	$\mathbf{1.0312^n}$	$\mathbf{1.0050^n}$	$\mathbf{1.0003^n}$	1.0953^n
3-Hitting Set	$\mathbf{1.1607^n}$	$\mathbf{1.1135^n}$	$\mathbf{1.0458^n}$	$\mathbf{1.0211^n}$	1.2405^n
4-Hitting Set	$\mathbf{1.2288^n}$	$\mathbf{1.1861^n}$	$\mathbf{1.1233^n}$	$\mathbf{1.0933^n}$	1.3229^n
Feedback Vertex Set	$\mathbf{1.2161^n}$	$\mathbf{1.1752^n}$	$\mathbf{1.0987^n}$	$\mathbf{1.0315^n}$	1.3081^n
Subset Feedback Vertex Set	$\mathbf{1.2288^n}$	$\mathbf{1.1861^n}$	$\mathbf{1.1233^n}$	$\mathbf{1.0933^n}$	1.3229^n
Cluster Vertex Deletion	$\mathbf{1.1318^n}$	$\mathbf{1.1047^n}$	$\mathbf{1.0676^n}$	$\mathbf{1.0315^n}$	1.2033^n
Proper Interval Vertex Deletion	$\mathbf{1.2558^n}$	$\mathbf{1.2097^n}$	$\mathbf{1.1404^n}$	$\mathbf{1.1066^n}$	1.3541^n
Multicut on Trees	$\mathbf{1.1034^n}$	$\mathbf{1.0817^n}$	$\mathbf{1.0525^n}$	$\mathbf{1.0315^n}$	1.1647^n

Table 4: Quantum β-approximation by Theorem 6.3, with bounded error, polynomial space and no quantum random access memory, rounded up. The last column is the quantum exact bound of Table 3, and for Vertex Cover the bound through Maximum Independent Set of [35]. For Multicut on Trees, n is the number of edges.

Every quantum exact algorithm is also a β-approximation, and every entry is below the quantum exact bound. To our knowledge, no quantum algorithm designed for exponential-time approximation of these problems has been stated before. Amplitude amplification over random sets of size $\beta \cdot \mathrm{OPT}$, the quantum form of the brute-force approximation of [13], gives 1.3097^n, 1.2580^n, 1.1769^n and 1.1360^n for the four ratios. The classical bounds of [13, 14] are the squares of the table entries or smaller, since they also use extension algorithms not covered by Theorem 6.3, such as the randomized 2.69998^k algorithm for Feedback Vertex Set.

The weighted form of approximate monotone local search [15] groups the elements into classes of weights within a factor $\gamma = 1 + \delta/2$, puts all classes far below the heaviest class of the optimum into the sample, and runs the unweighted scheme in each of the $O(\delta^{-1}\log(n/\delta))$ remaining classes with one extension procedure. The quantum form samples in every class inside one amplitude amplification, and it has these parts. The marked probability is at least the product of the hypergeometric probabilities of the classes, for the correct guess of the class sizes k_i of the optimum. Classes with $k_i > n_i/\zeta$ go entirely into the sample, where $\zeta = \beta/(1+\delta)$. The marking threshold is $\tau = w(W_k) + \zeta \sum_i \gamma^{i+1} k_i$, with W_k the part of the sample that is fixed by the guess, so that $\tau \le \beta\,\mathrm{OPT}_w$ at the correct guess. This gives the square root of the classical cost, with a subexponential factor for the guesses. The extension procedure may be a tree of kind (b′): a permissive tree with a weight threshold, which accepts on $(I, X, \tau - w(X), \ell)$ whenever a set of size at most ℓ within the threshold exists, and whose answers stay within the threshold. The weighted

Vertex Cover algorithm of [37] in polynomial space, with base 1.381, returns the first answer that is not NIL, so it is a tree of this kind, and its answers may be larger than ℓ. With the parts above, this gives a weighted 1.1-approximation for Vertex Cover in $O(1.0788^n)$ and, through Theorem 6.1, exact weighted Vertex Cover in $O(1.1296^n)$; weighted 3-Hitting Set gets $O(1.1607^n)$ at $\beta = 1.1$. The exact weighted bound $O^*(1.238^n)$ cited in [15] would give a smaller value, $\sqrt{1.238} < 1.1127$, if that algorithm is a backtracking tree.

7 Verification

The Lean library, built on Mathlib with only the standard axioms, proves the inequality $s\,x^{2(s-1)} \le y^{2s} \Rightarrow s^{1/(2s)}x'^{(s-1)/s} \le y$ for $0 \le x' \le x$, the bounds $\sqrt{1.6981} \le 1.3032$, $\sqrt{1.7159} \le 1.3100$ and $\sqrt{1.32703} \le 1.1520$, and the rows $k = 5$ above. A script checks the certificate of Theorem 3.2 in decimal arithmetic. Not formalized are the rules of [43], the terminal algorithms, the backtracking structure, the quantum walk, Theorem 9 of [36], Proposition 4.1 and the rows $k = 6$ to 18.

A second script runs the 4-colouring tree. On random states of graphs of maximum degree 4, one branching step at the weights of Theorem 3.2 has $\sum_j r^{\mu(P_j)-\mu(P)} \le 1$ in the real measure, and P is colourable if and only if a child is, on 27463 states that reach the cases of [43] for maximum degree at most 4. On 400 random terminal instances, (T1), (T2) and (T3) agree with brute force, with nine local lists per triangle in place of the blocks, and the blocks of Lemma 5.3 of [45] are checked with one and two triangles. On 300 random graphs with at most 16 vertices, the whole tree agrees with brute force and the terminal sizes add up to at most $r^{\mu(G,\emptyset,\emptyset)}$. The tree for 3-colouring of [5], the elimination of two-valued variables and the factor $e^{o(n)}$ are not run.

For the domination problems, the Lean files check $\sqrt{1.4969} \le 1.2235$, $\sqrt{1.4969^n} \le 1.2235^n$, $\sqrt{1.3226} \le 1.1501$ and $1.34165 \cdot 6^{1/2500000} \le 1.1583^2$. Scripts run the decision procedures of Section 5: Algorithm 4 of [33] with all its rules, the basic branching of [7] with marking and the rule for small components, and Algorithm 3 of [34], which has the same kinds of steps as its Algorithm 4. On 400, 400 and 300 random graphs with at most 15, 13 and 10 vertices, the smallest accepted budget equals the optimum computed by brute force in every case. The count of bad branchings is also checked on disjoint unions of regular graphs. The scripts run the unweighted procedures; the refined branchings of [7] and of Algorithm 4 of [34], the weighted rules and the bounds on the tree size are not run.

For Section 6, the Lean files prove the identity $2\sin x \sum_{j<m} \cos((2j+1)x) = \sin(2mx)$, the averaging bound $\sum_{j<m} \sin^2((2j+1)\theta) \ge m/4$ for $m \sin 2\theta \ge 1$, and the bases of Table 3. A script evaluates $\max_k \min_t \binom{n}{t}\binom{k}{t}^{-1} c^{k-t}$ against $(2-1/c)^n$, which it does not exceed, for $n \in \{50, 200, 1000, 3000\}$ and each c of the table. A second script runs the sampling and amplification with an exact classical oracle on random instances of 3-Hitting Set, Feedback Vertex Set and Cluster Vertex Deletion with up to 10 elements: clause branching, the branching of [21] with its matroid matching step replaced by further branching, and branching on induced paths with three vertices. On 120 instances each, the smallest accepted budget equals the optimum, at least $\binom{k'}{t}$ samples are marked at $k' = \mathrm{OPT}$, the larger of M/N and the averaged success probability is at least 0.416, where M of N samples are marked, and state-vector runs agree with the closed form. The walk, the bounded-error oracle, the algorithms of [38, 25, 22, 37, 9] and the tree sizes are not run. For Section 6.1, a script computes $\mathrm{best}(L, \beta)$ from the large-deviation exponent of the hypergeometric tail, reproduces the values 1.2072, 1.2099, 1.2068 and 1.4156 of [14] and 1.114 of [13], and checks that the exact cost at $n = 1000$ is within a factor $(n+1)^2$ of $\mathrm{best}(L, \beta)^n$ for every entry of Table 4; the maximization over k/n is numerical.

A classical version of the colouring recursion, with OR in place of quantum search, is compared

with brute force on random graphs with 5 to 9 vertices and $3 \leq k \leq 8$. Its base case for $k \leq 3$ respects the size limits strictly, at $k = 4$ it may ignore the limit, as the algorithm does, and the limits passed on are kept tight or enlarged at random. With either reduction chosen at random, and with Reduction 2 only, it agrees with brute force on 300 graphs each. As a control, lowering the limit for $G - S$ by one makes 80 of 300 answers wrong, so the test does depend on the limits.

8 Discussion

The gain for $k = 4$ comes from the branching algorithm of [43] and the terminal algorithms of [45] and carries through the reductions of [8, 36]. Any further classical improvement of 3-colouring or 4-colouring by a branching algorithm in polynomial space carries over in the same way, through the same computation. The exponents for $k \geq 6$ are upper bounds for the explicit procedure of Proposition 4.1. With finer grids and pieces they approach the values of the recursion (1), (2), which we expect but do not prove.

For the domination problems the quantum bounds are the square roots of the classical bounds used. They follow once the decision version of each classical algorithm is seen to be a backtracking tree, and any faster classical algorithm of this kind gives a faster quantum one in the same way.

For monotone local search the quantum bound is the square root of the classical one whenever the extension algorithm is a backtracking tree. Extension algorithms with steps that compute the optimum of a part before a tail call, such as the 2.0409^k algorithm for 3-Hitting Set [39], and the multivariate and sample-dependent forms of monotone local search, are not covered here. The same holds for approximate monotone local search, where randomized extension procedures that are not polynomial-time procedures, and deterministic approximate branching algorithms, are not covered.

Use of AI

The proofs, the code and the Lean files were developed with Claude Opus 5.5 (Anthropic), which was also used to search the literature, to check the proofs and to draft the text. The author is responsible for the content.

References

[1] M. Aprile, M. Drescher, S. Fiorini, T. Huynh. A tight approximation algorithm for the cluster vertex deletion problem. Mathematical Programming 197(2), 2023.

[2] V. Bafna, P. Berman, T. Fujito. A 2-approximation algorithm for the undirected feedback vertex set problem. SIAM Journal on Discrete Mathematics 12(3), 1999.

[3] M. Boyer, G. Brassard, P. Høyer, A. Tapp. Tight bounds on quantum searching. Fortschritte der Physik 46, 1998.

[4] G. Brassard, P. Høyer, M. Mosca, A. Tapp. Quantum amplitude amplification and estimation. Contemporary Mathematics 305, 2002.

[5] R. Beigel, D. Eppstein. 3-coloring in time $O(1.3289^n)$. Journal of Algorithms 54(2), 2005, arXiv:cs/0006046.

[6] N. Bourgeois, F. Della Croce, B. Escoffier, V. Th. Paschos. Fast algorithms for min independent dominating set. Discrete Applied Mathematics 161(4-5), 2013.

[7] N. Bourgeois, B. Escoffier, V. Th. Paschos. Fast algorithms for min independent dominating set. SIROCCO 2010, arXiv:0905.1993.

[8] J. M. Byskov. Enumerating maximal independent sets with applications to graph colouring. Operations Research Letters 32(6), 2004.

[9] Y. Cao. Unit interval editing is fixed-parameter tractable. ICALP 2015, arXiv:1504.04470.

[10] Y. Cao. Linear recognition of almost interval graphs. SODA 2016, arXiv:1403.1515.

[11] M. Cygan, M. Pilipczuk, M. Pilipczuk. On group feedback vertex set parameterized by the size of the cutset. Algorithmica 74, 2016, arXiv:1112.6255.

[12] M. Cygan, M. Pilipczuk, M. Pilipczuk, J. O. Wojtaszczyk. Subset feedback vertex set is fixed-parameter tractable. ICALP 2011.

[13] B. C. Esmer, A. Kulik, D. Marx, D. Neuen, R. Sharma. Faster exponential-time approximation algorithms using approximate monotone local search. ESA 2022, arXiv:2206.13481.

[14] B. C. Esmer, A. Kulik, D. Marx, D. Neuen, R. Sharma. Optimally repurposing existing algorithms to obtain exponential-time approximations. SODA 2024, arXiv:2306.15331.

[15] B. C. Esmer, A. Kulik, D. Marx, D. Neuen, R. Sharma. Approximate monotone local search for weighted problems. IPEC 2023, arXiv:2308.15306.

[16] G. Even, J. Naor, L. Zosin. An 8-approximation algorithm for the subset feedback vertex set problem. SIAM Journal on Computing 30(4), 2000.

[17] F. V. Fomin, S. Gaspers, D. Lokshtanov, S. Saurabh. Exact algorithms via monotone local search. Journal of the ACM 66(2), 2019, arXiv:1512.01621.

[18] F. V. Fomin, F. Grandoni, A. V. Pyatkin, A. A. Stepanov. Combinatorial bounds via measure and conquer: bounding minimal dominating sets and applications. ACM Transactions on Algorithms 5(1), 2008.

[19] M. Fürer. Solving NP-complete problems with quantum search. LATIN 2008.

[20] S. Gaspers, J. Z. Li. Quantum algorithms for graph coloring and other partitioning, covering, and packing problems. ICALP 2024, arXiv:2311.08042.

[21] Y. Iwata, Y. Kobayashi. Improved analysis of highest-degree branching for feedback vertex set. IPEC 2019, arXiv:1905.12233.

[22] Y. Iwata, M. Wahlström, Y. Yoshida. Half-integrality, LP-branching and FPT algorithms. SIAM Journal on Computing 45(4), 2016, arXiv:1310.2841.

[23] N. Garg, V. V. Vazirani, M. Yannakakis. Primal-dual approximation algorithms for integral flow and multicut in trees. Algorithmica 18, 1997.

[24] Y. Iwata. A faster algorithm for dominating set analyzed by the potential method. IPEC 2011, LNCS 7112.

[25] I. Kanj, G. Lin, T. Liu, W. Tong, G. Xia, J. Xu, B. Yang, F. Zhang, P. Zhang, B. Zhu. Algorithms for cut problems on trees. COCOA 2014, LNCS 8881, arXiv:1304.3653.

[26] A. Kulik, H. Shachnai. Analysis of two-variable recurrence relations with application to parameterized approximations. FOCS 2020, arXiv:1911.02653.

[27] E. L. Lawler. A note on the complexity of the chromatic number problem. Information Processing Letters 5(3), 1976.

[28] Kvantu algoritms maksimāla inducēta meža problēmai (a quantum algorithm for the maximum induced forest problem). Thesis, University of Latvia, `https://dspace.lu.lv/handle/7/66169`.

[29] L. Meijer. 3-Coloring in time $O(1.3217^n)$. arXiv:2302.13644, 2023.

[30] J. W. Moon, L. Moser. On cliques in graphs. Israel Journal of Mathematics 3, 1965.

[31] A. Montanaro. Quantum-walk speedup of backtracking algorithms. Theory of Computing 14(15), 2018, arXiv:1509.02374.

[32] I. Razgon. Exact computation of maximum induced forest. SWAT 2006, LNCS 4059.

[33] J. M. M. van Rooij, H. L. Bodlaender. Exact algorithms for dominating set. Discrete Applied Mathematics 159(17), 2011. Technical report UU-CS-2009-025.

[34] J. M. M. van Rooij, H. L. Bodlaender. Exact algorithms for edge domination. Algorithmica 64(4), 2012. Technical report UU-CS-2007-051.

[35] S. Senge. Faster quantum algorithms for vertex cover and independent set through recursive search with value calls. `https://github.com/ssenge/LAIF/tree/main/quantum-vertex-cover`, 2026.

[36] K. Shimizu, R. Mori. Exponential-time quantum algorithms for graph coloring problems. Algorithmica 84, 2022, arXiv:1907.00529.

[37] H. Shachnai, M. Zehavi. A multivariate approach for weighted FPT algorithms. ESA 2015; arXiv:1407.2033, A multivariate framework for weighted FPT algorithms.

[38] D. Tsur. Faster parameterized algorithm for cluster vertex deletion. arXiv:1901.07609.

[39] D. Tsur. Faster parameterized algorithm for 3-hitting set. arXiv:2501.06452.

[40] M. Wahlström. Algorithms, measures and upper bounds for satisfiability and related problems. PhD thesis, Linköping University, 2007.

[41] P. van 't Hof, Y. Villanger. Proper interval vertex deletion. Algorithmica 65(4), 2013.

[42] R. Wong, W.-L. Chang, W.-Y. Chung, A. V. Vasilakos. Biomolecular and quantum algorithms for the dominating set problem in arbitrary networks. Scientific Reports 13, 4205, 2023.

[43] P. Wu, H. Gu, H. Jiang, Z. Shao, J. Xu. A faster algorithm for the 4-coloring problem. ESA 2024, LIPIcs 308, 103:1–103:18.

[44] M. Xiao, H. Nagamochi. A refined exact algorithm for edge dominating set. Theoretical Computer Science 560, 2014.

[45] R. Yamano, T. Shibuya. Faster exact algorithms for 3- and 4-coloring: deterministic improvements and further randomized speedups. arXiv:2611.03115, 2026.